

Arquitectura de microservicios

UnoArena

Grupo 5

El dominio

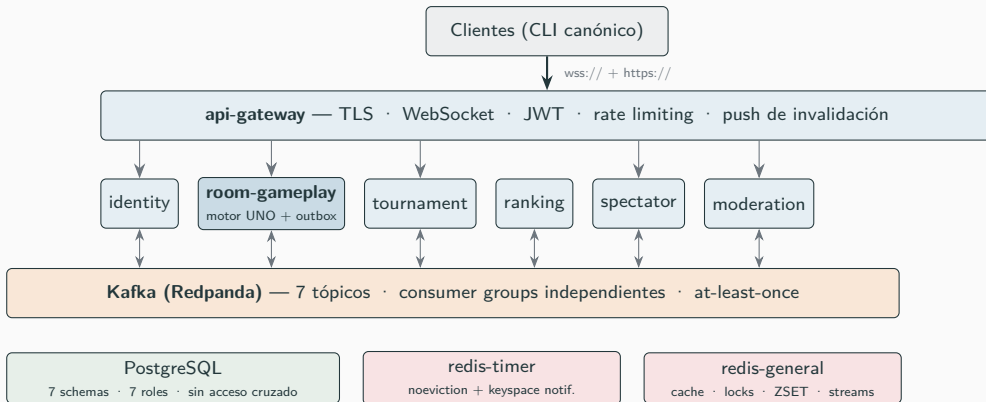
- UNO multijugador global, en tiempo real
- **Quick Play**: salas de 2–10 por matchmaking
- **Torneos de eliminación**: rondas Bo3, top 3 avanza

La escala de diseño

- Hasta **1.000.000** de jugadores por torneo
- Ronda 1 $\Rightarrow$ **$\sim$ 100.000 salas simultáneas**

El pico de la ronda 1 es el problema: casi todas las decisiones de arquitectura se justifican contra ese surge.

Arquitectura final



1 bounded context = 1 servicio = 1 schema con rol propio · despliegue completo con `deploy-all.sh` sobre kind

Bounded contexts: por qué estos siete

- **Room Gameplay:** núcleo write-heavy, row-lock por juego
- **Tournament:** orquestación Bo3
(“forfeit” = eliminación permanente, no salir del juego)
- **Ranking:** única autoridad de Elo
- **Spectator:** proyección read-only + ACL de privacidad
- **Identity:** sesión única, upstream de todos
- **Moderation:** audit-log, comandos correctivos

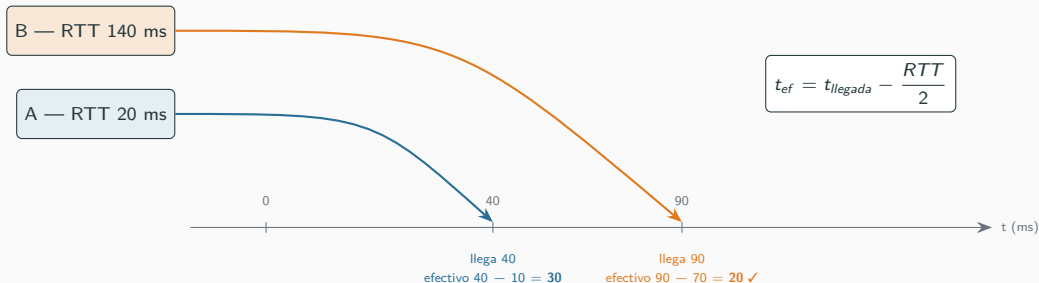
Lo que evaluamos y descartamos

- Matchmaking como servicio propio
→ cada modo ya tiene dueño natural

Cada contexto tiene consistencia y ciclo de vida de datos distintos $\Rightarrow$ sin base compartida.

Decisión: fairness entre regiones (lag compensation)

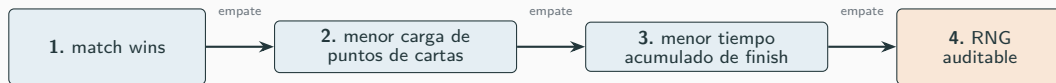
Problema: en JumpIn y ChallengeUno compiten varios jugadores; por orden de llegada, el más cercano al server siempre gana.



- RTT medido **por el servidor** vía heartbeats (rolling average) — el reloj del cliente jamás se usa.
- Ventana de carrera: **150 ms** de llegada. Empate dentro de ± 20 ms (o sin RTT medido) → RNG uniforme.
- Todo queda en el log: evento RaceResolved con RTTs y método — auditable, RTTs no expuestos a rivales.
- Misma suposición ($RTT/2$, ruteo simétrico) que la lag compensation de shooters como CS2.

Decisión: desempates para avanzar de ronda

Bo3 por sala de 10 → avanzan los **top 3**. Orden total:



- `finish_timestamp` con **reloj monotónico del server**; sin podio $\Rightarrow$ contribuye el timeout de 20 min.
- Forfeits: **debajo de todos los activos**, siempre. Sala con ≤ 3 activos $\Rightarrow$ avanzan todos.

Decisión: un único API Gateway como frontera pública

Qué hace — políticas, nunca lógica de dominio

- TLS y upgrade WS; único componente con interfaz pública
- JWT: firma verificada local + cache-aside
- Rate limiting: INCR por IP + ZSET sliding por usuario
- Fan-out de sala: registro `player_id` → WS; el 200 OK trae `notify_player_ids`

Alternativas evaluadas

✗ Sin gateway, cliente → cada servicio

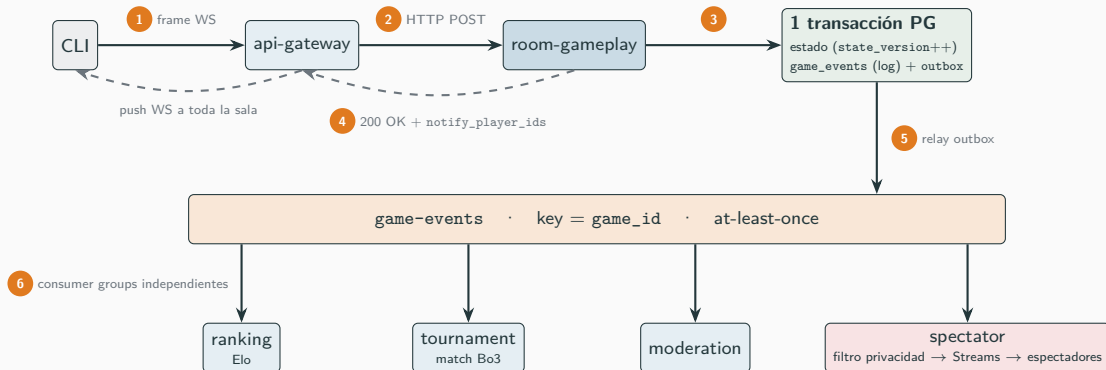
✓ Un gateway, tres públicos

(jugador WS · espectador WS · REST): registro unificado ⇒ el close frame alcanza *todas* las conexiones del jugador

Trade-offs aceptados

- Registro **in-memory por pod** ⇒ sticky routing por `game_id`; pod caído = reconexión
el estado vive en PG y Redis, no en el gateway
- Entrega con **1 réplica**: `/internal/push` caería en un pod al azar
remediación documentada: fan-out por Redis Pub/Sub

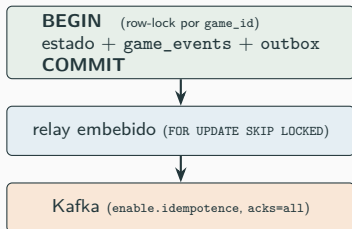
Flujo completo: jugar una carta



- El jugador ve su resultado **después del COMMIT**, sin esperar a Kafka → hot path < 100 ms (P99 < 200 ms).
- WebSocket y no SSE/polling: full-duplex en 1 conexión y *close frame* server-side para la sesión única.

Decisión: outbox transaccional (log-before-broadcast)

Regla: nadie observa un cambio de estado antes de que esté en el log durable.



Alternativas evaluadas

✗ Dual-write DB + Kafka:

crash entre escrituras = evento perdido o
broadcast falso

✗ Event sourcing puro:

rehidratar 400–1000 eventos por comando =
latencia

✓ Outbox: atomicidad gratis, estado JSONB → comando $O(1)$

Decisión: Kafka como backbone — tópicos, keys y particiones

¿Por qué Kafka? Consumer groups con lag independiente (aislar el burst), orden por partición, replay durable. RabbitMQ: sin fan-out con lag propio ni replay. Redis Streams: sin partition routing, memoria acotada.

Tópico	Partition key	Particiones (diseño / demo)	La key es la unidad de orden de...
game-events	game_id	100+ / 8	cada partida (spectator, torneo)
tournament-kickoff	room_id	100+ / 8	fan-out del surge
tournament-events	tournament_id	20 / 3	el bracket
identity-events	player_id	20 / 3	la sesión de un jugador
ranking-events	player_id	20 / 3	el Elo de un jugador
moderation-events	player_id/game_id	10 / 3	el sujeto del evento

Decisión: Redis — un rol distinto por contexto

Uso	Mecanismo
Timers (45 s / 5 s / 60 s / 20 min)	TTL + keyspace notif.
Invalidación de sesión	Pub/Sub → gateway
Fan-out a espectadores	Streams (XREAD)
Cache de sesión (JWT)	cache-aside, TTL 60 s
Rate limiting	INCR + ZSET sliding
Leaderboards Elo	Sorted Sets
Idempotencia / locks	SET NX PX

Dos instancias, políticas opuestas:

redis-timer = noeviction (un timer no se puede perder)

redis-general = cache/fan-out.

Timers: alternativas

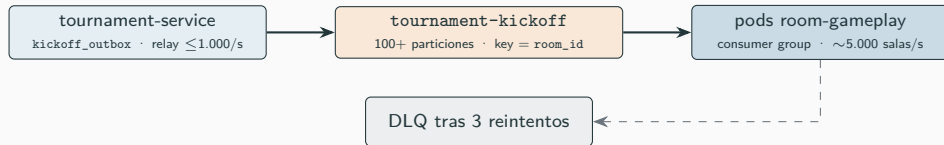
- ✗ in-process: no sobrevive un crash
- ✗ tabla PG + polling: 1 s de jitter en una ventana de 5 s
- ✓ Redis TTL: precisión ms, crash-safe, cancelación O(1)

Doble red de seguridad

- **Token fence**: UUID en la key y en el estado PG
→ expiración duplicada = no-op
- Sweeps de reconciliación (challenge cada 2 s) si se pierde la notificación

Principio: Redis nunca es fuente de verdad — es fast-path; la DB siempre tiene el fallback durable.

Decisión: el surge de kickoff (100 K salas en ≤ 120 s)



Alternativa evaluada

- ✗ Work queue en PG + workers HTTP:
backpressure artesanal, retry storms,
el endpoint HTTP como cuello de botella

Por qué Kafka fan-out

- El **consumer lag** es backpressure natural y observable
- Productor y consumers escalan por separado
- Room IDs **UUID5 determinísticos** → reintentos idempotentes

Métricas de negocio (se piden 3, hay 6)

- `uno_game_events_total` — 1 hook en el log = todo el dominio
- `uno_games_active` — gauge que se auto-repara (lee PG)
- `uno_matchmaking_queue_depth` — sube y cae a 0 al formar sala
- `uno_elo_updates_total` — casual / torneo / **reversión**
- `uno_websocket_connections_active` — jugador vs espectador
- `uno_rate_limit_hits` + `uno_session_events` — abuso y sesión única

Criterio: métricas sobre *el juego*, no sobre el proceso — lo que miraría un operador de torneo.

Demo: lo que van a ver ahora

1. Despliegue desde cluster vacío — `deploy-all.sh` (~2 min)

`kind` → 7 imágenes side-load → infra (PG, Redis ×2, Redpanda + tópicos) → observabilidad → 7 servicios; `kubectl get pods -n unoarena` para verlos corriendo

2. Tres terminales — `register` + login: **bob**, **alice** y **carol** (espectadora)

una sesión por terminal: `UNOARENA_SESSION_FILE` distinto en cada una

3. Partida casual 1v1 — bob y alice a la cola de matchmaking:

`play / draw / uno / challenge`, WILD con color, 409 stale + reconciliación

4. Espectadora en vivo — carol: `room list` → `spectate <roomId>`

conteos sí, manos jamás: la ACL de privacidad se aplica al consumir el evento

5. Torneo con 20 bots → bracket Bo3 real, campeón y Elo de torneo

en Grafana: la cola de matchmaking, los eventos/s y las escrituras de Elo moviéndose en vivo

6. Teardown — `deploy-all.sh --destroy`

¡Muchas gracias!